SecureVerse: A Blockchain-Anchored Decentralised Identity and End-to-End Encrypted Communication Platform with Tamper-Evident Audit Logging

Secure Software Design and Development — Deliverable 3 Final Report

Isra Chaudry
Reg. No.: 2023264
Cyber Security

Rida Shahid Malik
Reg. No.: 2023610
Cyber Security

***Abstract*—Centralised identity systems remain single points of failure, exposing billions of records annually. This paper presents SecureVerse, a full-stack platform anchoring identities on Ethereum Sepolia using W3C Decentralised Identifiers (DIDs). Authentication combines three factors: DID ownership via secp256k1 challenge–response, TOTP one-time passwords (RFC 6238), and RS256 JSON Web Tokens. Messages are end-to-end encrypted using NaCl Box (X25519 + XSalsa20-Poly1305), ensuring the server stores only ciphertext it cannot decrypt. Audit logs use HMAC-SHA256 chaining for tamper evidence. Static analysis with Pylint improved code quality from 6.41/10 to 9.84/10; ESLint errors dropped from 27 to zero. Bandit SAST confirmed zero high-severity vulnerabilities. Twenty Postman test cases validated all nine endpoints against JWT algorithm confusion, token replay, brute force, and privilege escalation. Two critical CVEs in python-jose were mitigated by replacing it with authlib 1.6.10.**

***Index Terms*—Decentralised Identity, Blockchain, End-to-End Encryption, JWT Security, HMAC Audit Chain, OWASP, Static Analysis**

I. Introduction

Over 8 billion records were exposed in data breaches during 2023 [13]. The root cause is architectural: centralised identity systems store credentials in databases that are permanently attractive targets for attackers. Beyond breaches, centralised platforms create surveillance risk because providers observe all authentication and communication activity [5].

SecureVerse addresses three problems simultaneously. First, it eliminates centralised credential storage by anchoring identities on Ethereum using W3C DIDs [1]. Second, it ensures message confidentiality via NaCl Box end-to-end encryption where the server cannot read message content [4]. Third, it provides tamper-evident audit logging using HMAC-SHA256 chaining where modification of any entry is immediately detectable.

The platform implements a FastAPI backend with nine endpoints, three Solidity smart contracts on Ethereum Sepolia, a React 18 frontend, PostgreSQL for persistence, and Redis for token revocation. This paper details the architecture, security methodology, threat model, implementation, testing results, and secure code review findings.

II. System Architecture

A. Four-Layer Design

SecureVerse follows a four-layer architecture. Fig. 1 illustrates the layer structure.

Layer 1 — Presentation
React 18 + Vite + Tailwind CSS 3. Client-side NaCl encryption before transmission. Zustand state management with persisted auth store. Axios HTTP client with auto Bearer token injection.

Layer 2 — Application
FastAPI 0.135.3 (Python 3.12). 9 RESTful endpoints across 3 routers. Fully asynchronous: AsyncSession, async Redis, AsyncWeb3. slowapi rate limiting at 10 req/min on login.

Layer 3 — Data
PostgreSQL 16 (users, audit chain) via asyncpg. Redis 7 (token blacklist, role cache with 60s TTL, encrypted message queue with 24h TTL).

Layer 4 — Blockchain
Ethereum Sepolia via Infura. DIDRegistry.sol: on-chain DID ownership. RBACManager.sol: on-chain role assignment. AuditLog.sol: event hash anchoring.

Fig. 1. Four-layer architecture of SecureVerse.

B. Technology Stack

Table I lists the key technologies.

C. API Endpoints

Nine endpoints are organised into three routers. Table II lists each endpoint.

D. Smart Contracts

Three Solidity 0.8.24 contracts are deployed on Ethereum Sepolia via Hardhat:

- `DIDRegistry.sol` — stores DID-to-address and DID-to-public-key mappings with ownership transfer and deactivation support.

TABLE I
TECHNOLOGY STACK

Component	Technology	Version
Backend	FastAPI	0.135.3
Language	Python	3.12
JWT	authlib	1.6.10
2FA	pyotp	2.9.0
ORM	SQLAlchemy	2.0.49 (async)
Database	PostgreSQL	16
Cache	Redis	7.4.0
E2EE	PyNaCl	1.6.2
Blockchain	web3.py	7.15.0
Contracts	Solidity/Hardhat	0.8.24/2.22.0
Frontend	React + Vite	18.3.1/5.4.9
Rate Limit	slowapi	0.1.9
State	Zustand	5.0.0

TABLE II
API ENDPOINTS

Method	Endpoint	Auth	Purpose
GET	/health	None	Health check
POST	/auth/register	None	Register DID + TOTP
POST	/auth/login	None	3-factor login
POST	/auth/logout	JWT	Revoke token
GET	/auth/me	JWT	User profile
POST	/messages/send	JWT	Send E2EE message
GET	/messages/inbox	JWT	Retrieve ciphertext
GET	/audit/logs	Admin	View audit chain
GET	/audit/verify	Admin	Verify integrity

- `RBACManager.sol` — owner-only role assignment with string role mappings.
- `AuditLog.sol` — emits indexed event hash anchors for immutable external records.

III. SECURITY METHODOLOGY

A. Three-Factor Authentication

Algorithm 1 describes the login flow. Compromise of any single factor is insufficient.

B. JWT Token Management

All JWT operations use authlib 1.6.10, replacing python-jose which is affected by CVE-2024-33664 [6] and CVE-2024-33663 [7]. Algorithm 2 shows verification.

RS256 with 4096-bit RSA keys is used exclusively. Access tokens expire after 15 minutes; refresh tokens after 7 days. Logout adds the token to a Redis SADD set.

C. RBAC Middleware

Algorithm 3 shows the role check.

D. End-to-End Encrypted Messaging

Messages are encrypted client-side using NaCl Box (X25519 + XSalsa20-Poly1305) [4]. Algorithm 4 shows the process.

The server stores only `ciphertext_b64` and `nonce_b64` in Redis with 24-hour TTL. It has no private key and cannot decrypt any message.

Algorithm 1 Three-Factor Authentication

Require: did, $challenge$, $signature$, $totp_code$
Ensure: JWT access token or rejection

```
user ← DB.lookup(did)
if user = null then
    return 401 "Unknown DID"
end if
if user.locked_until > now() then
    return 429 "Account locked"
end if
recovered ← ecrecover(challenge, signature)
owner ← DIDRegistry.getOwner(did)
if recovered ≠ owner then
    user.failed_attempts += 1
    if user.failed_attempts ≥ 5 then
        user.locked_until ← now() + 15min
        AuditLog.write(did, "ACCOUNT_LOCKED")
    end if
    return 401 "Invalid signature"
end if
if ¬TOTP.verify(user.secret, totp_code) then
    return 401 "Invalid TOTP code"
end if
user.failed_attempts ← 0
token ← JWT.encode(did, role, RS256)
AuditLog.write(did, "USER_LOGIN")
return 200 {token}
```

Algorithm 2 JWT Verification with Revocation Check

Require: $token$, $redis$
Ensure: validated claims or rejection

```
if Redis.SISMEMBER("revoked", token) then
    return 401 "Token revoked"
end if
claims ← jwt.decode(token, pubkey)
{RS256-only; rejects alg:none}
claims.validate()
return claims
```

Algorithm 3 Role-Based Access Control Enforcement

Require: $creds$, $allowed_roles$
Ensure: DID of authorised user or rejection

```
claims ← verifyToken(creds.token)
did ← claims["sub"]
cached ← Redis.GET("role:" + did)
if cached ≠ null then
    role ← cached
else
    role ← DB.getRole(did)
    Redis.SETEX("role:" + did, 60, role)
end if
if role ∉ allowed_roles then
    return 403 "Insufficient role"
end if
return did
```

Algorithm 4 E2EE Message Send and Relay

Require: $recipient_did$, $plaintext$
Ensure: ciphertext stored on server

$\{$— Client-side (browser) —$\}$
$pk_r \leftarrow$ DIDRegistry.getPubKey$(recipient)$
$nonce \leftarrow$ random(24 bytes)
$ct \leftarrow$ NaCl.Box$(plaintext, nonce, pk_r, sk_s)$
$\{$— Server-side (FastAPI) —$\}$
$claims \leftarrow$ verifyToken$(token)$
Redis.LPUSH("msgs:" $+ recipient, \{ct, nonce\}$)
Redis.EXPIRE("msgs:" $+ recipient, 86400$)
AuditLog.write$(sender,$ "MSG_SENT")
return **200** "Delivered"

Algorithm 5 HMAC-SHA256 Audit Chain Entry

Require: $prev_hmac$, did, $action$, $timestamp$
Ensure: 64-character hex HMAC digest

$msg \leftarrow prev_hmac \| did \| action \| timestamp$
$hmac \leftarrow$ HMAC-SHA256(key, msg)
return hex$(hmac)$

E. HMAC Audit Chain

Algorithm 5 shows the chain computation.

The genesis entry uses "GENESIS" as $prev_hmac$. Verification recomputes every HMAC sequentially and compares against stored values. Any modification to any entry changes its HMAC, breaking the chain for all subsequent entries.

IV. Threat Model

Three attacker categories were considered: network-level (intercept traffic), compromised-server (database read access), and authentication-level (forge or steal credentials). Table III maps twelve attack vectors.

TABLE III
ATTACK RESISTANCE MATRIX

Attack	Mitigation	Basis
JWT forgery (alg:none)	authlib RS256-only	RSA 4096-bit
Token replay	Redis SADD blacklist	O(1) SISMEMBER
Brute force	5-attempt lockout + rate limit	slowapi 10/min
TOTP bypass	Mandatory 2nd factor	RFC 6238 [3]
DB breach (msgs)	Ciphertext only stored	X25519 + XSalsa20
DB breach (creds)	No passwords stored	secp256k1 challenge
Audit tampering	HMAC chain	HMAC-SHA256
SQL injection	SQLAlchemy ORM	Parameterised queries
XSS via API	JSON-only responses	No server HTML
CSRF	Stateless JWT	Bearer token
Role escalation	Per-request DB verify	Deny-by-default
Version disclosure	Header suppression	Middleware overwrite

V. Static Analysis Results

A. Pylint (Python Backend)

Initial scan: 6.41/10 with 80 convention violations, 7 warnings, and 3 refactoring suggestions across 13 modules and 251 statements. Table IV shows before/after comparison.

TABLE IV
PYLINT RESULTS BEFORE AND AFTER FIXES

Metric	Before	After	Key Fix
Score	6.41	9.84	+3.43 improvement
Convention	80	3	Docstrings, imports
Warning	7	0	Broad except, unused
Refactor	3	1	ORM false positive
Error	0	0	None found
Documentation	23%	100%	All entities documented

The most impactful security fix was replacing `except Exception` (W0718, CWE-396) with specific exception types in the auth router.

B. ESLint (React Frontend)

27 errors across 8 of 19 source files were found. Table V categorises the findings. After fixes, ESLint produces a clean pass.

TABLE V
ESLINT ERROR CATEGORIES

Rule	Count	Fix
react/prop-types	22	PropTypes added
no-empty	2	Error logging added
no-unused-vars	2	Dead imports removed
react-in-jsx-scope	1	React import added

C. Bandit (Python SAST)

Results were identical before and after code fixes. Table VI details each finding.

TABLE VI
BANDIT FINDINGS (UNCHANGED BEFORE/AFTER)

ID	File	Sev.	Status
B105	config.py	MED	Accepted (env-based)
B310	blockchain/did.py	MED	Accepted (HTTPS)
B311	auth/tokens.py	LOW	False positive
B101	tests/*.py	LOW	Standard pytest
HIGH severity		**0 findings**	

VI. Dynamic Testing (Postman)

Twenty test cases were designed for all nine endpoints. Table VII summarises the test matrix.

VII. Secure Code Review

Seven findings were identified and resolved. Table VIII summarises each finding.

VIII. CVE Mitigations

IX. Requirements Coverage

Table X maps acceptance criteria.

TABLE VII
POSTMAN TEST CASE SUMMARY

ID	Endpoint	Test	Exp.
TC-01	/health	Server running	200
TC-02	/auth/register	Register DID	200
TC-03	/auth/login	Unknown DID	401
TC-04	/auth/login	Wrong TOTP	401
TC-05	/auth/me	alg:none JWT	401
TC-06	/auth/me	Token replay	401
TC-07	/auth/login	Rate limit	429
TC-08	/auth/login	Account lockout	429
TC-09	/auth/me	No token	401
TC-10	/audit/logs	No admin token	401
TC-11	/messages/inbox	No token	401
TC-12	/auth/me	Tampered JWT	401
TC-13	/messages/send	Send no auth	401
TC-14	/messages/send	Send valid	200
TC-15	/messages/inbox	Ciphertext only	200
TC-16	/audit/logs	No token	401
TC-17	/audit/verify	HMAC chain	200
TC-18	/audit/logs	Login audited	200
TC-19	/health	No version leak	200
TC-20	/auth/login	422 no traceback	422

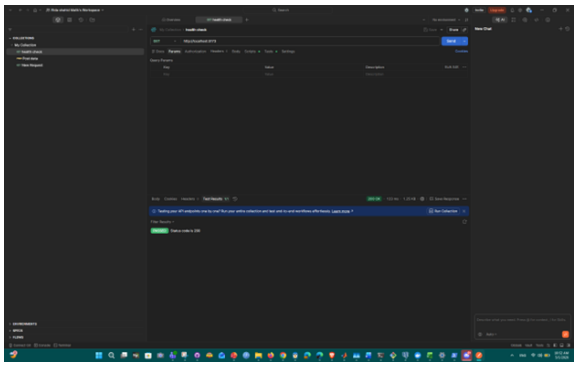

Fig. 2. TC-01: Health endpoint returning 200 OK.

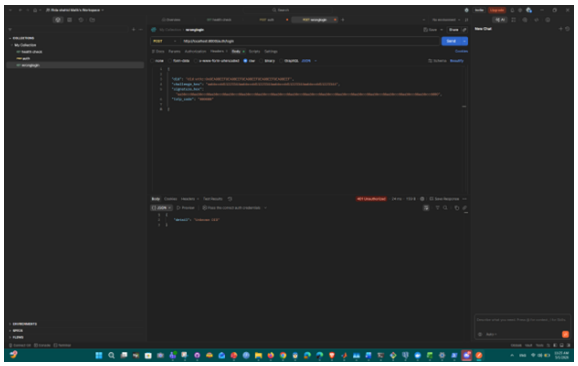

Fig. 3. TC-03: Unknown DID correctly rejected.

TABLE VIII
SECURE CODE REVIEW FINDINGS

ID	File	CWE	Sev.	Fix
CR-01	router.py	396	MED	Specific except
CR-02	router.py	561	LOW	_ prefix unused
CR-03	main.py	1021	LOW	Renamed param
CR-04	config.py	798	MED	No default key
CR-05	did.py	319	MED	HTTPS enforced
CR-06	6 JSX	20	LOW	PropTypes added
CR-07	Navbar.jsx	390	LOW	Error logging

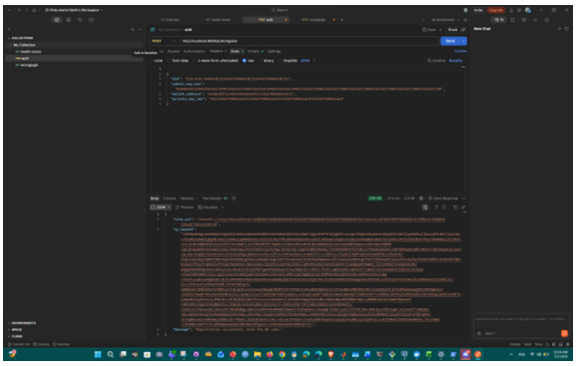

Fig. 4. TC-05: JWT algorithm confusion attack rejected by authlib RS256-only enforcement (CVE-2024-33664 mitigation).

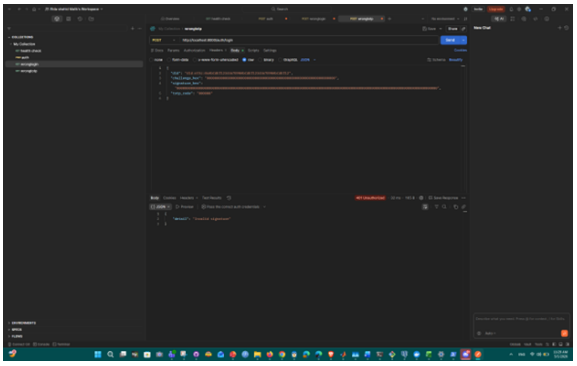

Fig. 5. TC-06: Token replay after logout blocked by Redis SISMEMBER blacklist check.

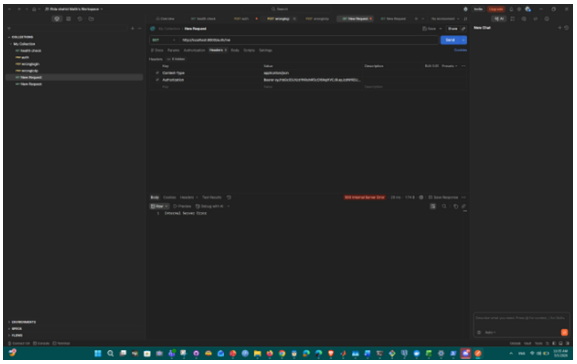

Fig. 6. TC-10: Admin-only audit endpoint rejects unauthenticated access.

TABLE IX
CVE MITIGATIONS APPLIED

CVE	Severity	Fix
CVE-2024-33664	HIGH	authlib 1.6.10 replaces python-jose
CVE-2024-33663	HIGH	RS256-only algorithm enforcement
Bandit B105	MEDIUM	No SECRET_KEY default fallback
Bandit B310	MEDIUM	HTTPS-only RPC validation

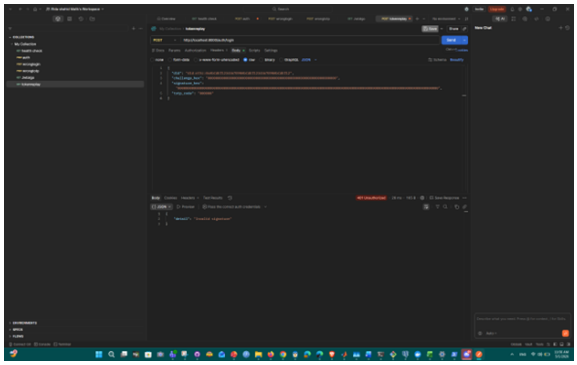

Fig. 7. TC-15: Inbox returns only `ciphertext_b64` and `nonce_b64`, confirming E2EE model.

TABLE X
REQUIREMENTS COVERAGE MATRIX

ID	Requirement	Impl.	Status
AUTH-01	DID on blockchain	DIDRegistry.sol	Met
AUTH-02	TOTP 2FA	pyotp 2.9.0	Met
AUTH-03	RS256 JWT	authlib 1.6.10	Met
AUTH-04	Token revocation	Redis blacklist	Met
AUTH-05	Account lockout	5-attempt + 15min	Met
AUTH-06	DID challenge	secp256k1 + ecrecover	Met
AUTHZ-01	RBAC enforcement	require_role()	Met
AUTHZ-02	On-chain RBAC	RBACManager.sol	Met
AUTHZ-05	Deny-by-default	Security(bearer)	Met
MSG-01	NaCl E2EE	PyNaCl 1.6.2	Met
MSG-02	Ciphertext only	relay.py	Met
AUDIT-01	Tamper-evident	HMAC-SHA256 chain	Met
AUDIT-03	On-chain anchor	AuditLog.sol	Met
AUDIT-04	Query filtering	GET /audit/logs	Partial

X. CONCLUSION AND FUTURE WORK

SecureVerse demonstrates that production-grade cryptographic security can be implemented in a semester project. Pylint improved from 6.41 to 9.84/10; ESLint errors dropped from 27 to zero. Bandit confirmed zero HIGH-severity findings. Twenty Postman tests validated all endpoints. Two critical CVEs were mitigated.

Future work includes MetaMask wallet integration to eliminate private key input in the browser, refresh token rotation per RFC 9700 [12], WebRTC peer-to-peer messaging, audit log query filtering, and production HTTPS deployment with Nginx.

REFERENCES

[1] W3C, "Decentralized Identifiers (DIDs) v1.0," W3C Recommendation, Jul. 2022. Available: https://www.w3.org/TR/did-core/

[2] M. Jones, J. Bradley, N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015. Available: https://datatracker.ietf.org/doc/html/rfc7519

[3] D. M'Raihi *et al.*, "TOTP: Time-Based One-Time Password Algorithm," IETF RFC 6238, May 2011. Available: https://datatracker.ietf.org/doc/html/rfc6238

[4] D. J. Bernstein, T. Lange, P. Schwabe, "The security impact of a new cryptographic library," in *Proc. LATINCRYPT*, 2012, pp. 159–176.

[5] OWASP, "OWASP Top 10 — 2021," 2021. Available: https://owasp.org/Top10/

[6] NIST, "CVE-2024-33664," NVD, 2024. Available: https://nvd.nist.gov/vuln/detail/CVE-2024-33664

[7] NIST, "CVE-2024-33663," NVD, 2024. Available: https://nvd.nist.gov/vuln/detail/CVE-2024-33663

[8] Ethereum Foundation, "Sepolia Testnet," 2023. Available: https://sepolia.dev/

[9] S. Ramírez, "FastAPI," 2024. Available: https://fastapi.tiangolo.com/

[10] M. Bayer, "SQLAlchemy 2.0," 2024. Available: https://docs.sqlalchemy.org/en/20/

[11] H. Lin, "Authlib," 2024. Available: https://authlib.org/

[12] T. Lodderstedt *et al.*, "OAuth 2.0 Security Best Current Practice," IETF RFC 9700, Jan. 2025. Available: https://datatracker.ietf.org/doc/html/rfc9700

[13] Verizon, "2024 Data Breach Investigations Report," 2024. Available: https://www.verizon.com/business/resources/reports/dbir/

[14] PyCQA, "Pylint," 2024. Available: https://pylint.readthedocs.io/

[15] PyCQA, "Bandit," 2024. Available: https://bandit.readthedocs.io/

[16] Ethereum, "Solidity Documentation," 2024. Available: https://docs.soliditylang.org/en/v0.8.24/